

Computer Networks

Redes de Computadores

RC

Teórica SETE

LEI

João Leitão

©2026 NOVA FCT

Aula Sete

- Da Janela Deslizante no caso do TCP
- O Segmento TCP
- O Three-Way Handshake
- Terminar uma Ligação
- Terminar Abruptamente: RST
- Controlo de Fluxo, Revisitado

A Semana Passada

- Na última aula, apresentámos os números de sequência indexados ao byte do TCP, um Número de Sequência Inicial aleatório, e o campo AdvertisedWindow — todos relevantes para o formato do segmento TCP.
- A janela deslizante resolve a entrega fiável e ordenada, mas como é que se deve iniciar uma ligação, terminá-la, temporizar as retransmissões, e os detalhes do controlo de fluxo?
- **Hoje e na próxima aula:** as respostas do TCP para cada uma dessas questões.

O Mesmo Mecanismo, Nova Função

Conclusão:
tudo hoje continua a utilizar Last Acknowledge Received (LAR), Last Frame Sent (LFS), Receive Window Size (RWS) — exatamente a mesma informação de controlo da janela deslizante que estudámos na última aula. O TCP apenas acrescenta um handshake para começar, uma troca de FIN/ACK para terminar, e — como veremos na próxima aula — um timeout adaptativo em vez de um timeout fixo.

Revisão: Onde Ficámos

- **SeqNum indexado a byte:** cada número de sequência do TCP conta bytes no fluxo, não pacotes.
- **Número de Sequência Inicial aleatório:** escolhido de forma a que um segmento atrasado de uma ligação anterior, no mesmo par de portas, não possa ser confundido com segmentos da nova ligação.
- **AdvertisedWindow:** o tamanho da janela do próprio recetor, transportado em cada segmento — o controlo de fluxo reutilizando os mesmos mecanismos da janela deslizante.

O Segmento TCP, Revelado

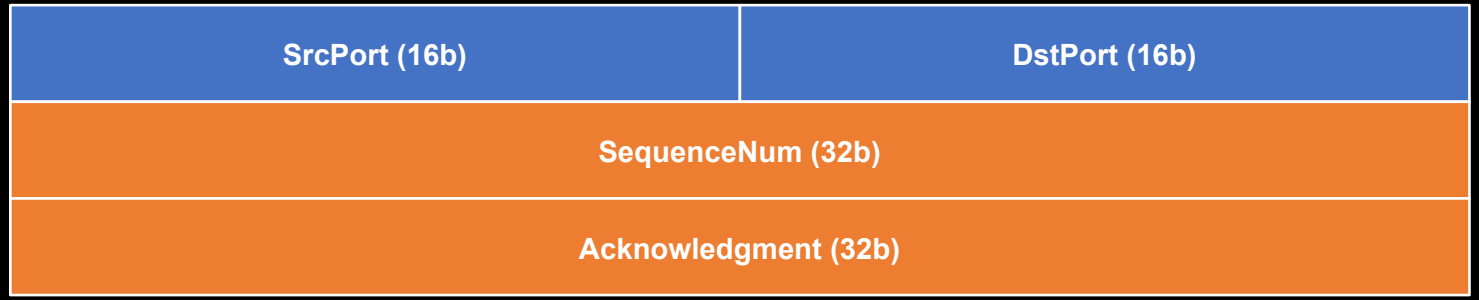


SrcPort e DstPort — a mesma informação de controlo que encontramos no cabeçalho UDP para multiplexar e desmultiplexar várias aplicações entre máquinas.

Quem Está a Falar com Quem

- **SrcPort e DstPort, mais os endereços IP de origem e destino (presentes no cabeçalho IP),** formam em conjunto a chave de desmultiplexagem do TCP — o tuplo de 4 elementos (SrcPort, SrcIPAddr, DstPort, DstIPAddr) identifica univocamente uma ligação.
- **As mesmas duas portas podem ser reutilizadas mais tarde por uma ligação TCP completamente diferente —** é por isso que o Número de Sequência Inicial precisa de ser imprevisível, e não apenas incrementado a partir de 0, para que não se confundam fluxos TCP de duas ligações distintas que, por acaso, tenham a mesma chave de desmultiplexagem.

O Segmento TCP, Revelado

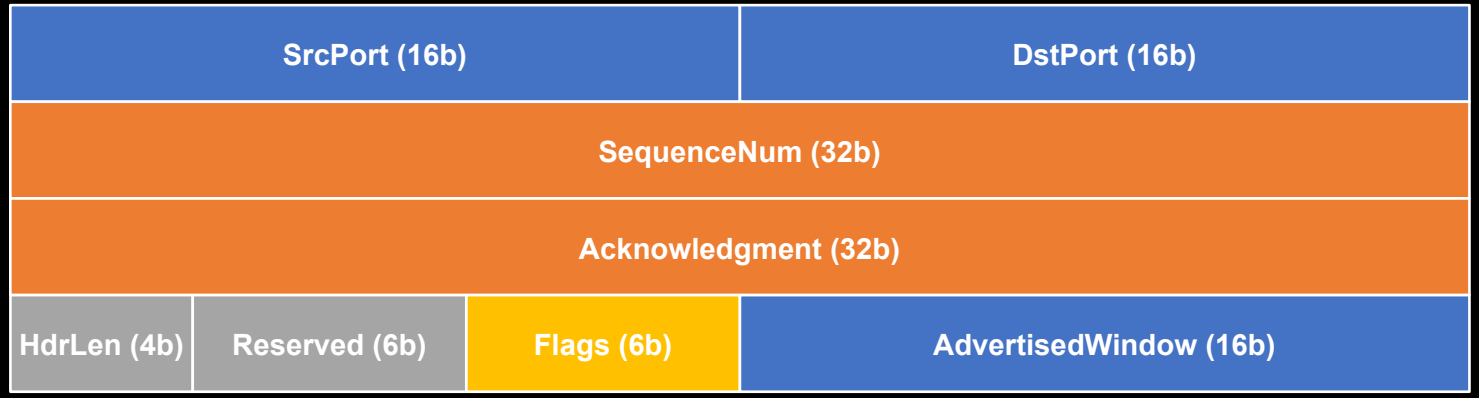


Ambos os campos, em cada segmento — o TCP é full-duplex, ambos os lados enviam e confirmam ao mesmo tempo.

Ambas as Direções, em Cada Segmento

- **O TCP é full-duplex:** cada lado é, simultaneamente, emissor e recetor do seu próprio fluxo de bytes, na mesma ligação.
- **SequenceNum indica onde começam os dados DESTE segmento** no próprio fluxo de bytes do emissor — contando bytes, não segmentos, exatamente como a janela deslizante da última aula.
- **O Acknowledgment viaja sem custo:** indica quanto do fluxo do OUTRO lado já chegou até agora — é transportado juntamente com os dados que, por acaso, sigam nesta direção, em vez de um segmento separado só com ACK.

O Segmento TCP, Revelado



Reserved (6 bits): não utilizado, tem de ser zero — reservado para o que o TCP venha a precisar de acrescentar mais tarde.

Seis Bits, Seis Significados

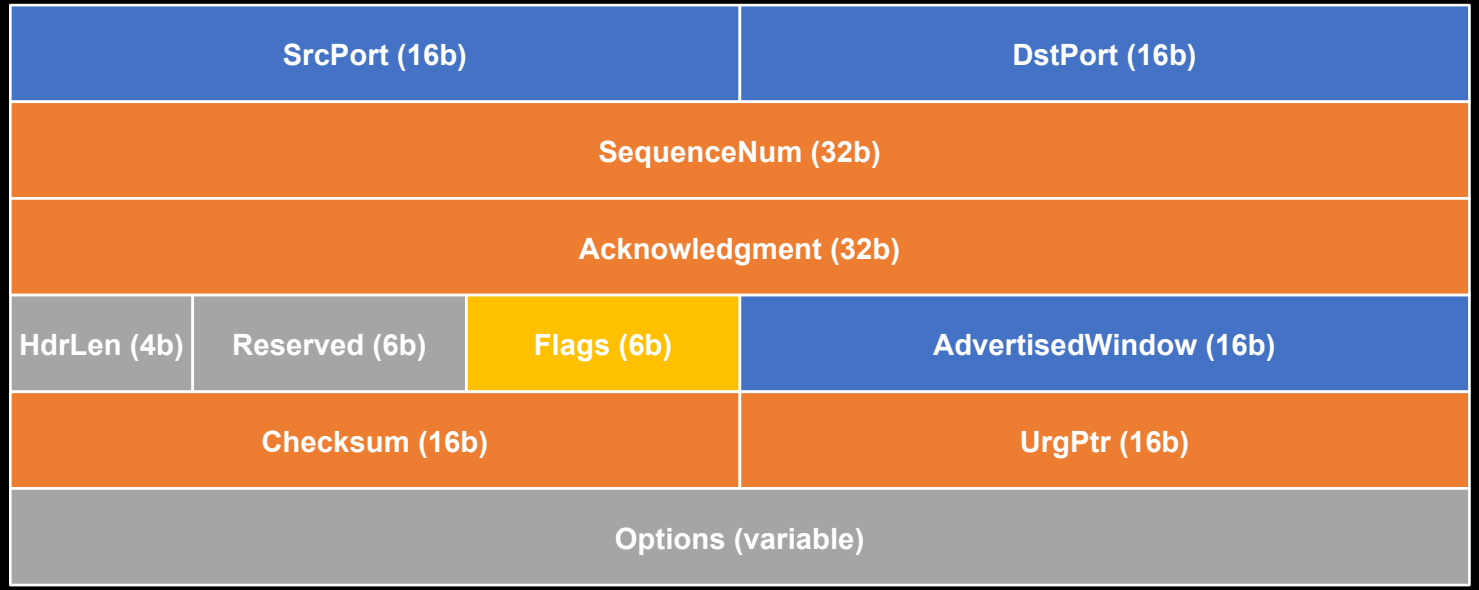
- **Um segmento não transporta apenas dados — também transporta informação de controlo:** estará a abrir uma ligação? A fechar uma? Apenas a confirmar dados? Seis bits de flags condensam tudo isso num único campo.
- **SYN / FIN:** usados ao estabelecer e terminar uma ligação.
- **ACK:** ativo sempre que o campo Acknowledgment é válido — verdade para quase todos os segmentos depois de a ligação estar aberta.
  - **RESET:** o recetor está confuso — por exemplo, chegou um segmento para uma ligação que não reconhece — e aborta-a. Mais sobre isto assim que virmos como é suposto uma ligação terminar.
- **URG / PUSH:** dados urgentes, e um pedido para entregar imediatamente à aplicação os bytes em buffer — ambos pensados para aplicações de terminal interativas, onde cada tecla premida importava; **raramente usados hoje em dia**.

O Campo Flags, Bit a Bit

URG	ACK	PSH	RST	SYN	FIN
dados urgentes	ACK válido	enviar agora	abortar	abrir	fechar

A ordem dos bits é fixada pela especificação do TCP (RFC 793).

O Segmento TCP Completo



HdrLen *mede o comprimento do cabeçalho em palavras de 32 bits — necessário porque o campo Options o pode estender.*

Reserved *são 6 bits reservados para futuras extensões do protocolo (devem ser enviados com valor zero)*

Flags *são 6 bits usados para transmitir informação de controlo em segmentos específicos.*

Porquê uma Fase de Estabelcer Ligação Explícita?

- Um protocolo ao nível da ligação física liga sempre os mesmos dois computadores — nunca houve dúvida sobre quem estava do outro lado. O TCP é diferente: suporta uma ligação lógica entre quaisquer dois processos, em qualquer ponto da Internet.
- Por isso, antes de quaisquer dados da aplicação serem transmitidos, os dois lados têm de concordar explicitamente em comunicar um com o outro.

Abertura Ativa e Passiva

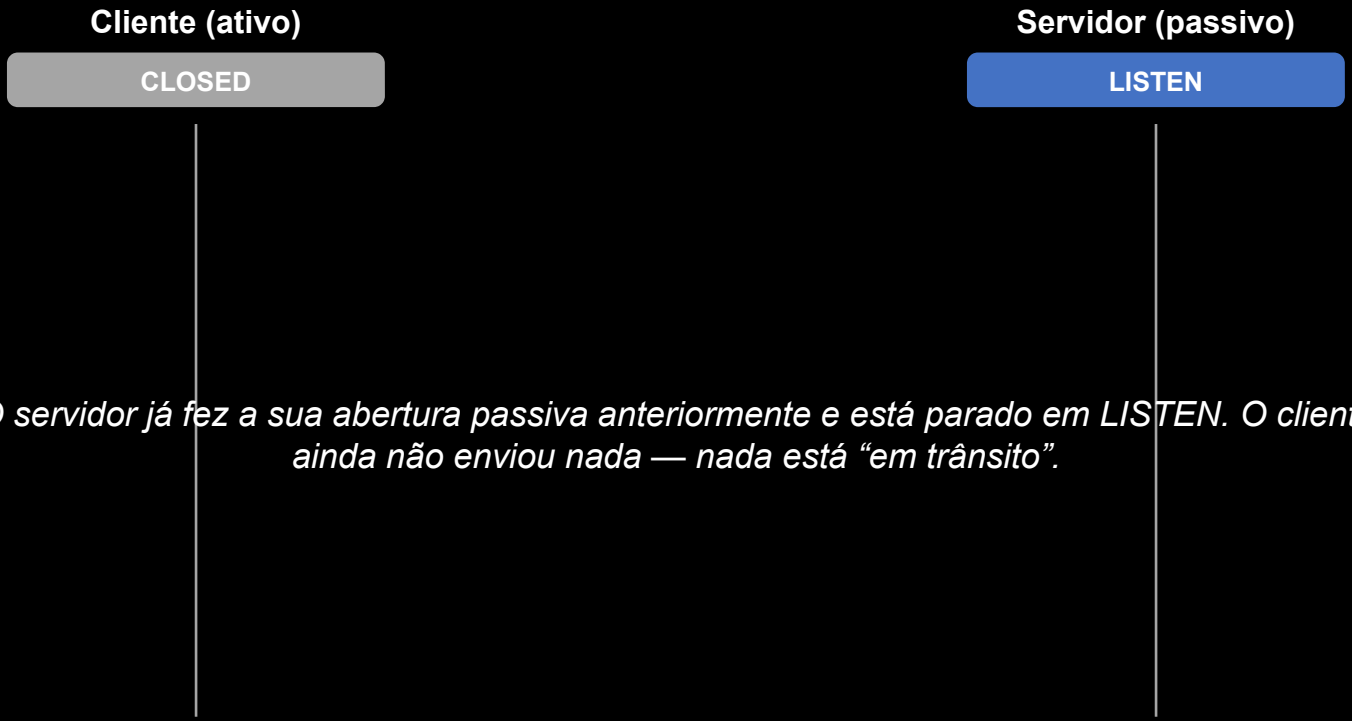
- **O estabelecimento da ligação é assimétrico:** um dos lados faz uma abertura ativa (normalmente o cliente), o outro uma abertura passiva (normalmente o servidor) — como marcar um número de telefone, ou atender a chamada.

- **Também faz mais do que apenas "dizer olá":** esta troca estabelece o estado partilhado necessário para a janela deslizante funcionar — ambos os lados precisam de saber o número de sequência inicial do outro antes de qualquer um poder enviar dados.

A Ver o Processo Acontecer: Mensagem a Mensagem

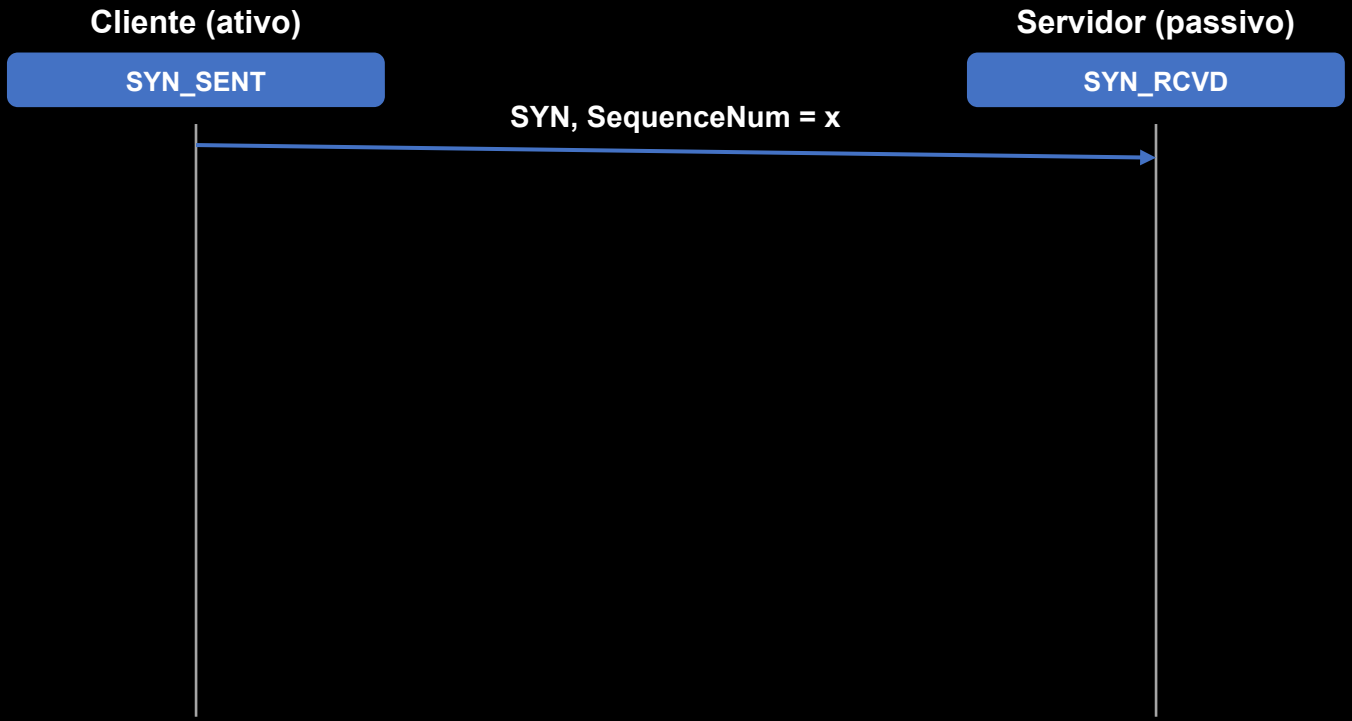
- O processo para iniciar uma ligação TCP chama-se three-way handshake; este nome sugere como o mecanismo funciona: há um total de três mensagens trocadas entre os dois extremos da ligação, a primeira das quais é enviada pelo cliente (que toma a iniciativa de iniciar a ligação).
- Vamos seguir as três mensagens — e, para cada uma, tanto o que transporta na rede como o que provoca no estado interno do TCP de cada lado.

Antes de Qualquer Coisa Ser Enviada



O servidor já fez a sua abertura passiva anteriormente e está parado em LISTEN. O cliente ainda não enviou nada — nada está “em trânsito”.

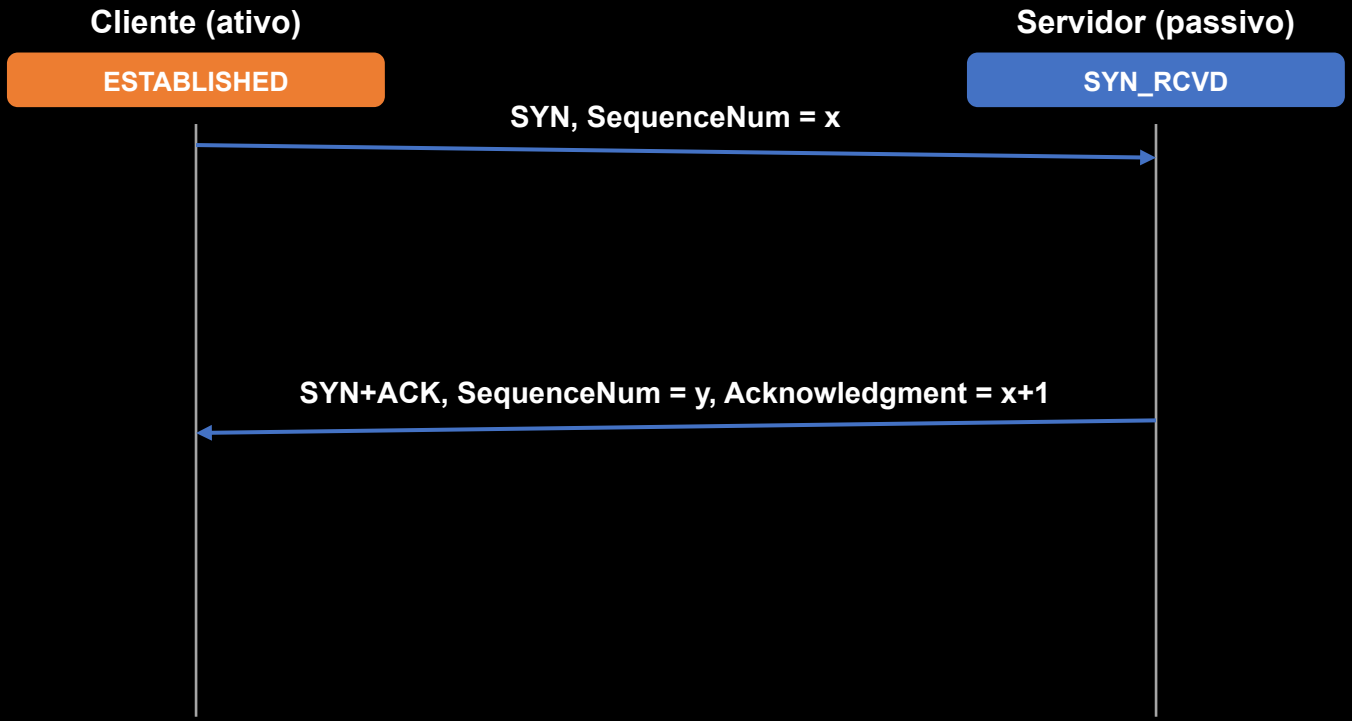
O Three-Way Handshake



Uma Mensagem Depois

- O cliente declara o seu próprio número de sequência inicial (Flags = SYN, SequenceNum = x) e move-se para o estado SYN_SENT no instante em que o envia.
- Do lado do servidor, a chegada desse mesmo segmento é o evento que o move de LISTEN para SYN_RCVD — um participante passivo, à espera, torna-se um participante ativo.

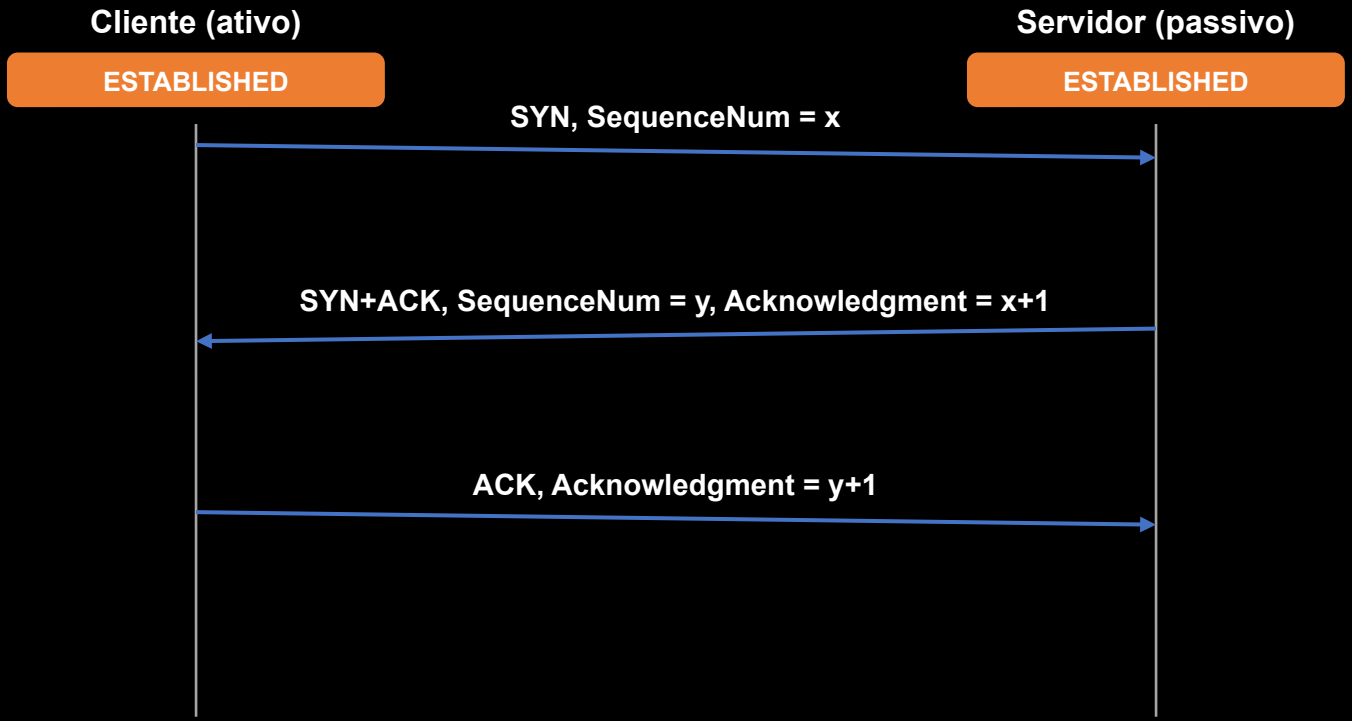
O Three-Way Handshake



O Servidor Responde e o Cliente Avança

- A resposta do servidor faz duas tarefas num único segmento: confirma o número de sequência do cliente (Ack = x+1) e declara o próprio número de sequência inicial do servidor (SYN, SequenceNum = y).
- Eis a parte subtil: o cliente passa diretamente para ESTABLISHED no momento em que recebe este segmento — não espera que o seu próprio ACK (o terceiro e último passo) seja confirmado. Enviar esse ACK e entrar em ESTABLISHED acontecem juntos, como uma única transição.

O Three-Way Handshake



A Interpretar o Handshake

- **A terceira mensagem fecha o ciclo para o servidor:** só quando este ACK final chega é que o estado do próprio servidor passa de SYN_RCVD para ESTABLISHED.

- **Porque não começar ambos os lados em 0?** Um ISN aleatório protege contra um segmento atrasado de uma ligação TCP anterior, que use as mesmas chaves de desmultiplexagem (i.e., o mesmo tuplo endereço IP de origem, porta de origem, endereço IP de destino, porta de destino), ser confundido com uma nova ligação TCP.

Um Exemplo Concreto

- O cliente escolhe x = 15000 aleatoriamente e envia SYN, SequenceNum = 15000.
- O servidor escolhe y = 8500000 e responde SYN+ACK, SequenceNum = 8500000, Acknowledgment = 15001.
- O cliente responde ACK, Acknowledgment = 8500001 — ambos os lados concordam agora onde começa o seu próprio fluxo de bytes.

A Mesma História vista como Máquina de Estados

- Eis a mesma troca outra vez, mas desenhada do ponto de vista de “em que estado está cada lado?” em vez de “o que vai na rede?” — as mesmas três mensagens, as mesmas duas transições que cada lado faz.

Abrir uma Ligação

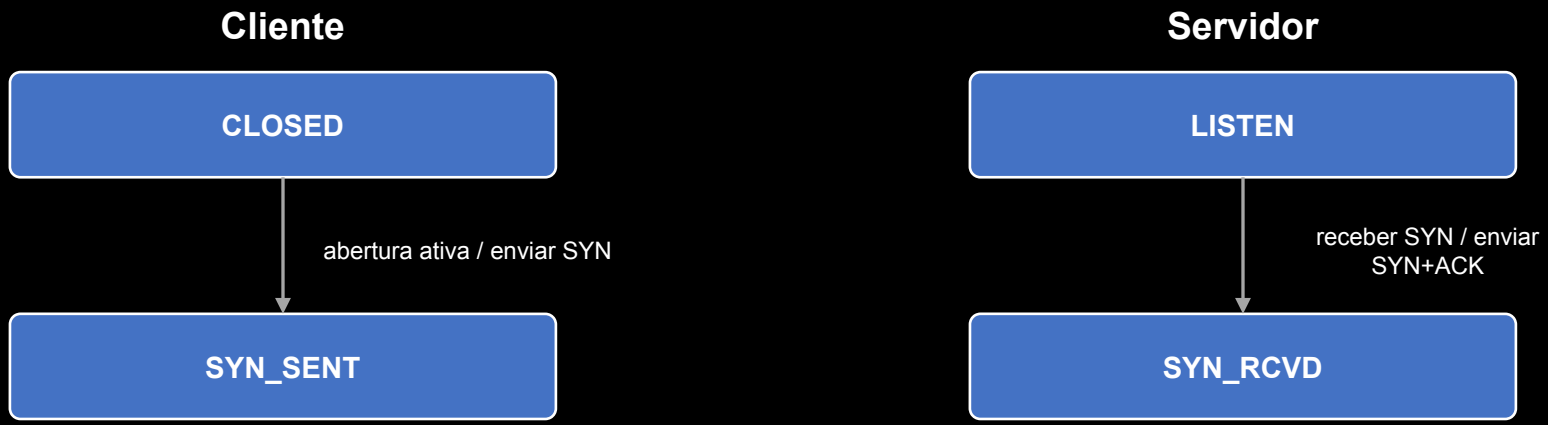
Cliente

CLOSED

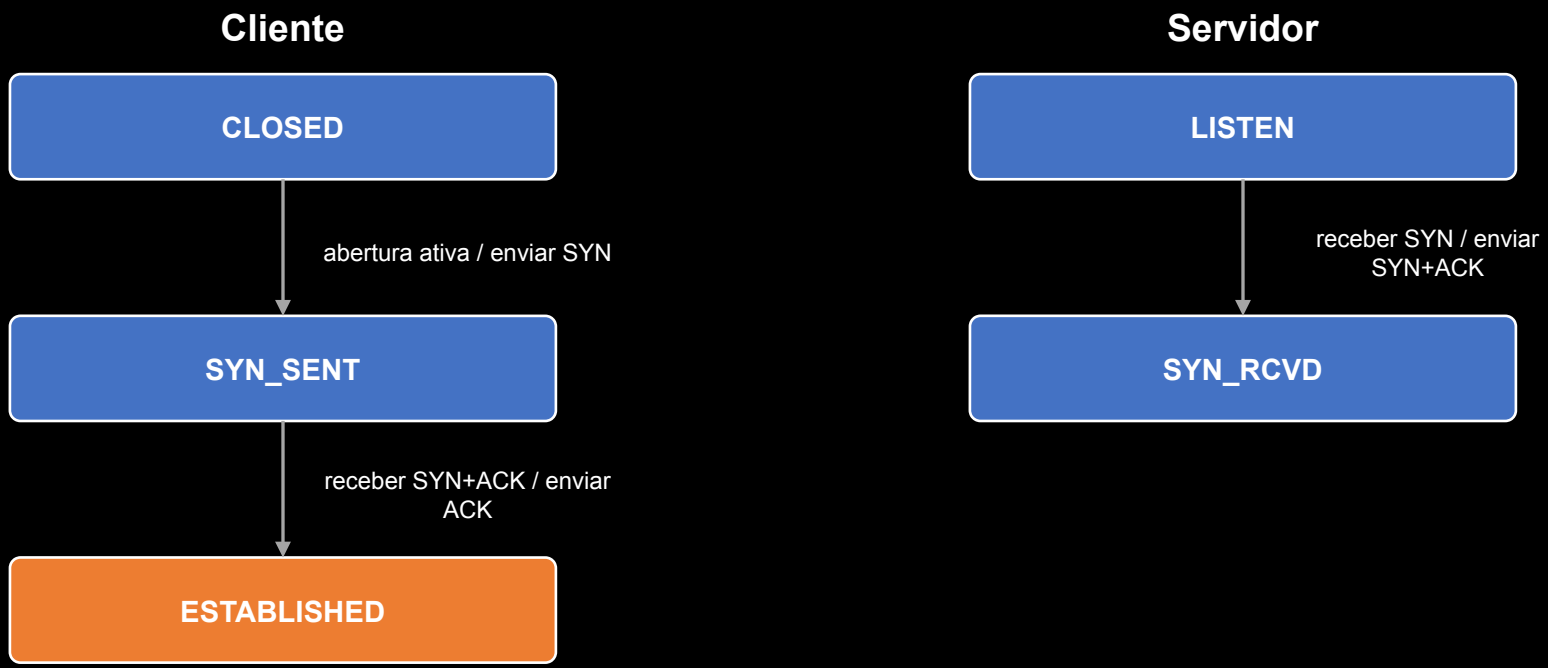
Servidor

LISTEN

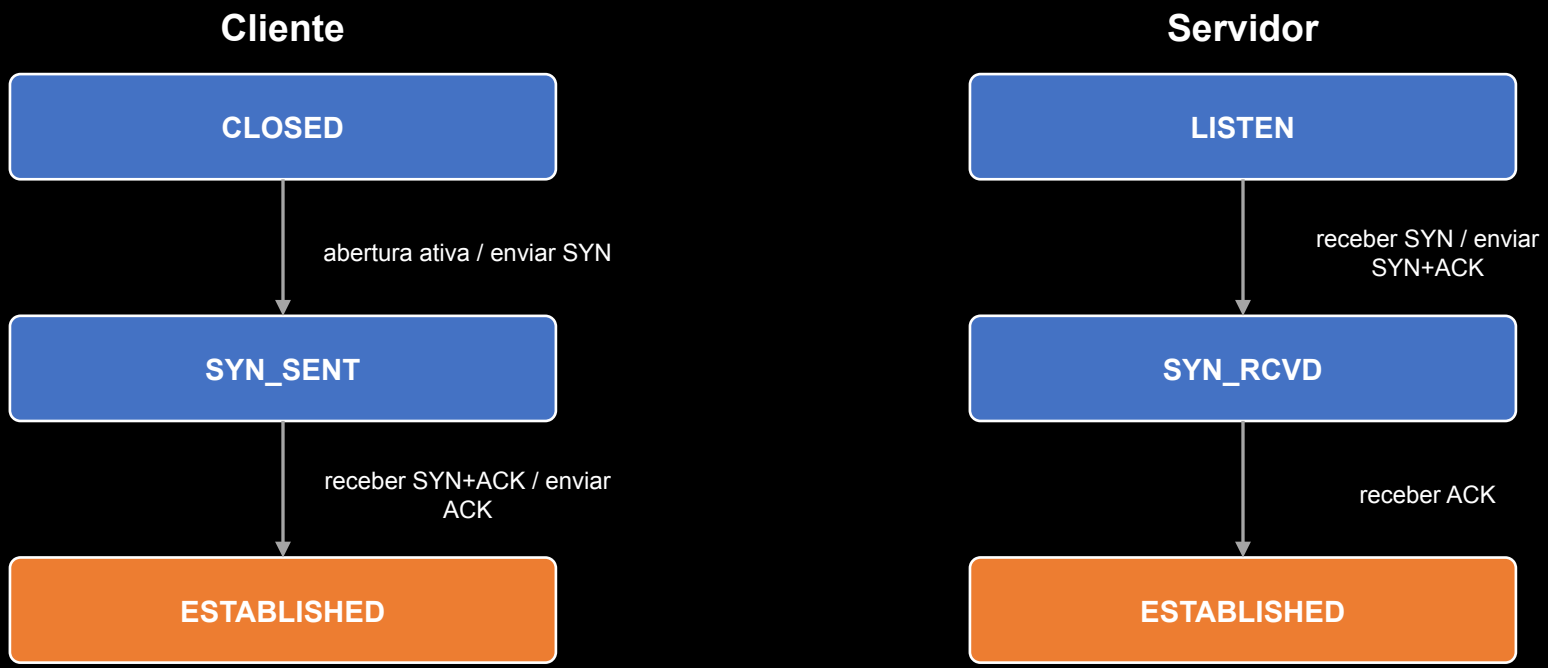
Abrir uma Ligação



Abrir uma Ligação



Abrir uma Ligação



Um Socket por Cliente

- **A única função do socket de escuta é esperar pela próxima ligação.** Se o accept() devolvesse esse MESMO socket para a própria conversa, o servidor ficaria sem nada com que escutar — nenhum socket livre para aceitar um segundo cliente até o primeiro terminar.

- **Como ultrapassar esta limitação:** accept(): devolve um novo socket correspondente a esta ligação acabada de estabelecer — um socket dedicado a este único cliente.
- **O socket original mantém-se intocado:** “quando accept retorna, o socket original continua a existir e está no estado listen; é usado em invocações futuras de accept” — já livre para aceitar a próxima ligação.

Um Socket por Cliente

Nota:
O socket devolvido pelo accept (ligado ao extremo do cliente) está associado ao mesmo IP e PORTA que o socket original usado para aceitar a ligação do cliente.
É por isso que a chave de demux do TCP inclui o IP e a PORTA do cliente (porque pode haver vários sockets do lado do servidor associados ao mesmo IP e PORTA).

Um Handshake, Três Mensagens

Conclusão:
o three-way handshake não é apenas uma forma educada de dois extremos se apresentarem — é a troca mínima necessária para que ambos os lados conheçam o número de sequência inicial do outro antes de qualquer dado ser enviado, para que todos os outros mecanismos funcionem.

Fechar uma Ligação

- Abrir uma ligação tem um funcionamento simples e direto: uma abertura passiva (servidor), uma abertura ativa (cliente), três mensagens. Será que fechar uma funciona da mesma forma?

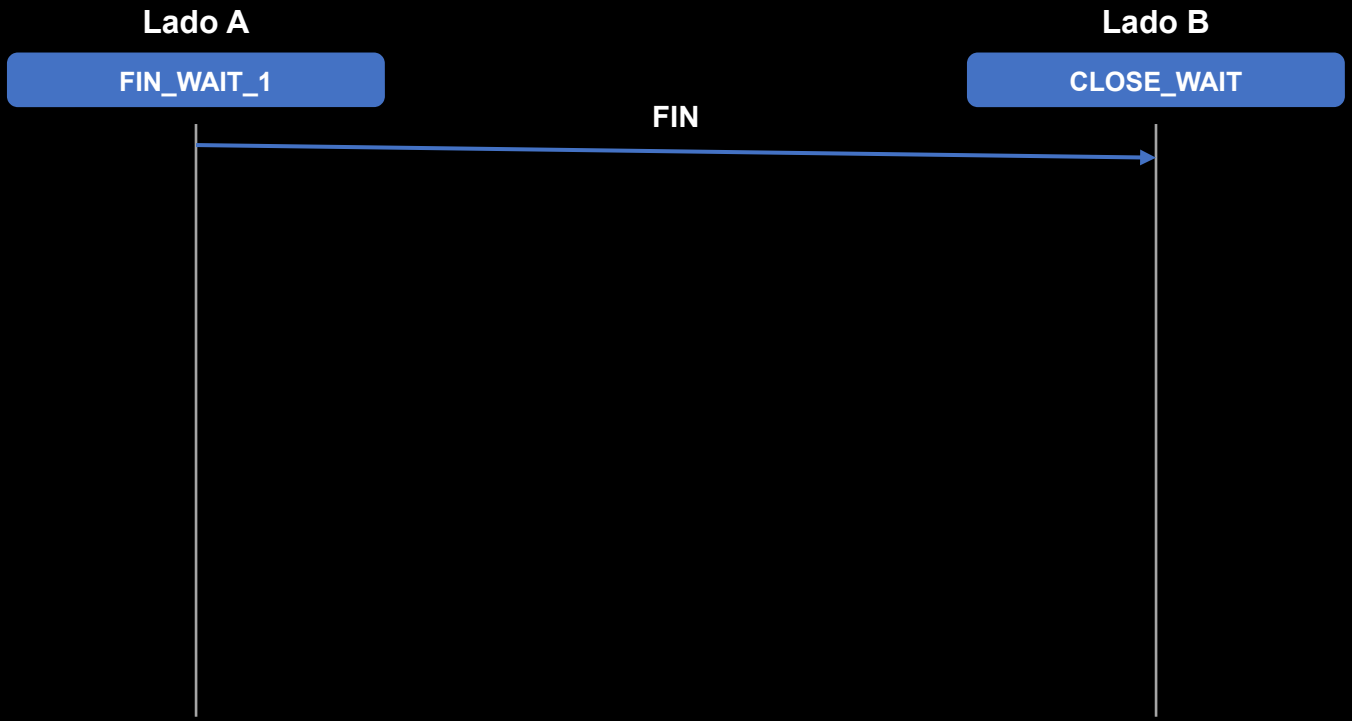
Fechar uma Ligação

- Abrir uma ligação tem um funcionamento simples e direto: uma abertura passiva (servidor), uma abertura ativa (cliente), três mensagens. Será que fechar uma funciona da mesma forma?
- **O estabelecimento é assimétrico** (uma abertura ativa, uma abertura passiva) — o encerramento não é: cada lado tem de fechar independentemente a sua própria metade da ligação.
- Como a aplicação de um dos lados pode ainda ter dados para enviar depois de o outro lado dizer que terminou, uma direção pode ser fechada enquanto a outra permanece aberta.

A Observar o Caminho Mais Comum

- Há três formas de uma ligação fechar — vamos ver as três — mas comecemos por ver a mais comum, mensagem a mensagem, exatamente como fizemos para o handshake.

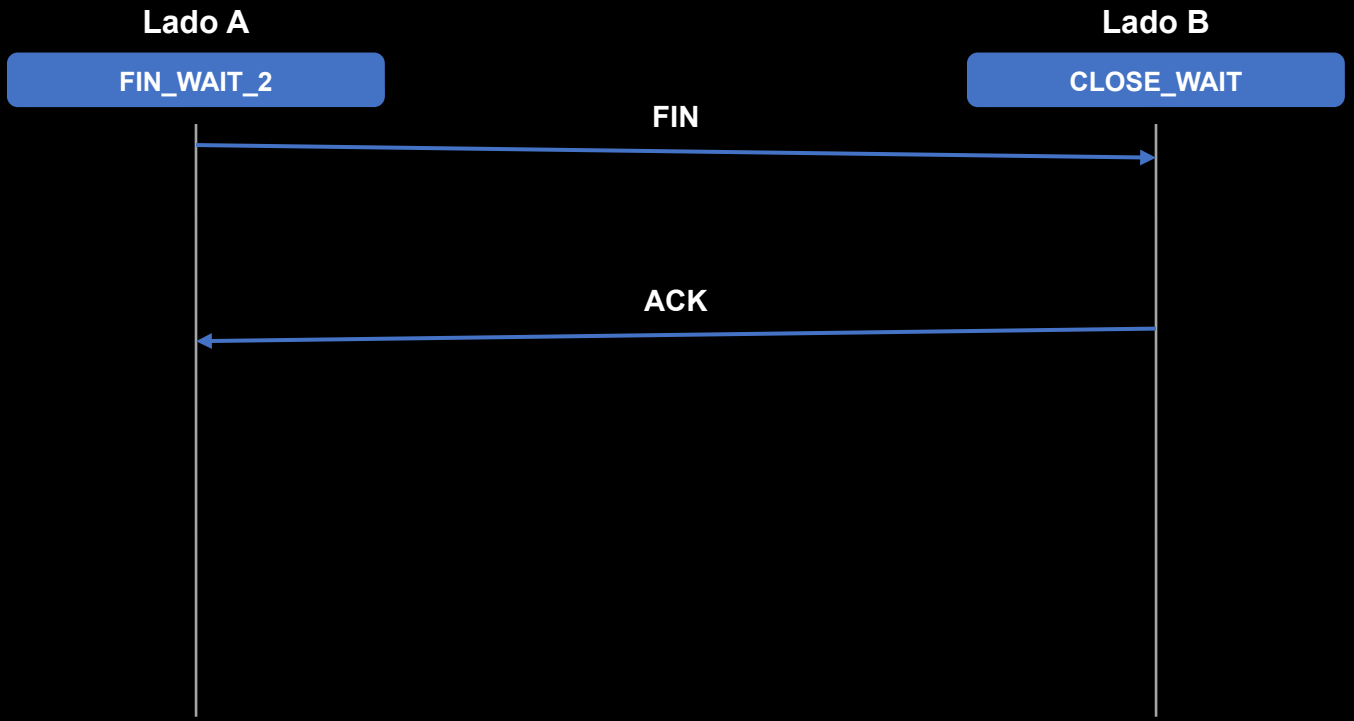
O Lado A Fecha Primeiro



Um dos Lados Começa a Sair

- Enviar um FIN move o Lado A do estado ESTABLISHED para FIN_WAIT_1 imediatamente. O outro lado, ao recebê-lo, move-se do estado ESTABLISHED para CLOSE_WAIT e envia automaticamente um ACK de volta — a sua aplicação pode ainda ter dados por enviar.

O Lado A Fecha Primeiro



O Lado A Fecha Primeiro



O Lado B Também Termina, Finalmente

- Quando a sua própria aplicação termina, o Lado B envia o seu próprio FIN (até esse ponto a aplicação pode ainda enviar dados) e move-se do estado CLOSE_WAIT para LAST_ACK.
- Receber esse FIN move o Lado A do estado FIN_WAIT_2 para TIME_WAIT — já viu tudo o que precisava, mas ainda não pode terminar a ligação.

O Lado A Fecha Primeiro



Caminho Dois: O Lado B Fecha Primeiro

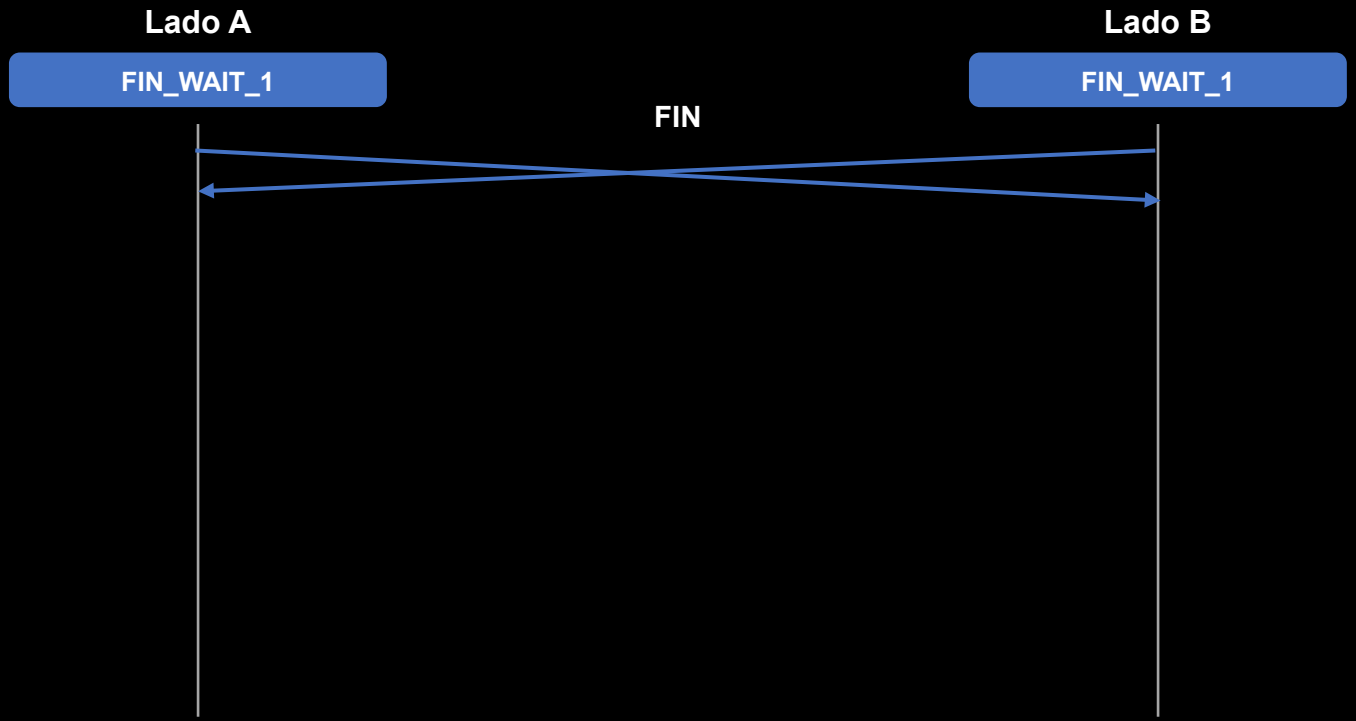
- Sem surpresa, exatamente as mesmas transições que vimos para quando o Lado A fecha primeiro — apenas com os papéis do Lado A e do Lado B trocados

Caminho Três: Ambos os Lados Fecham ao Mesmo Tempo

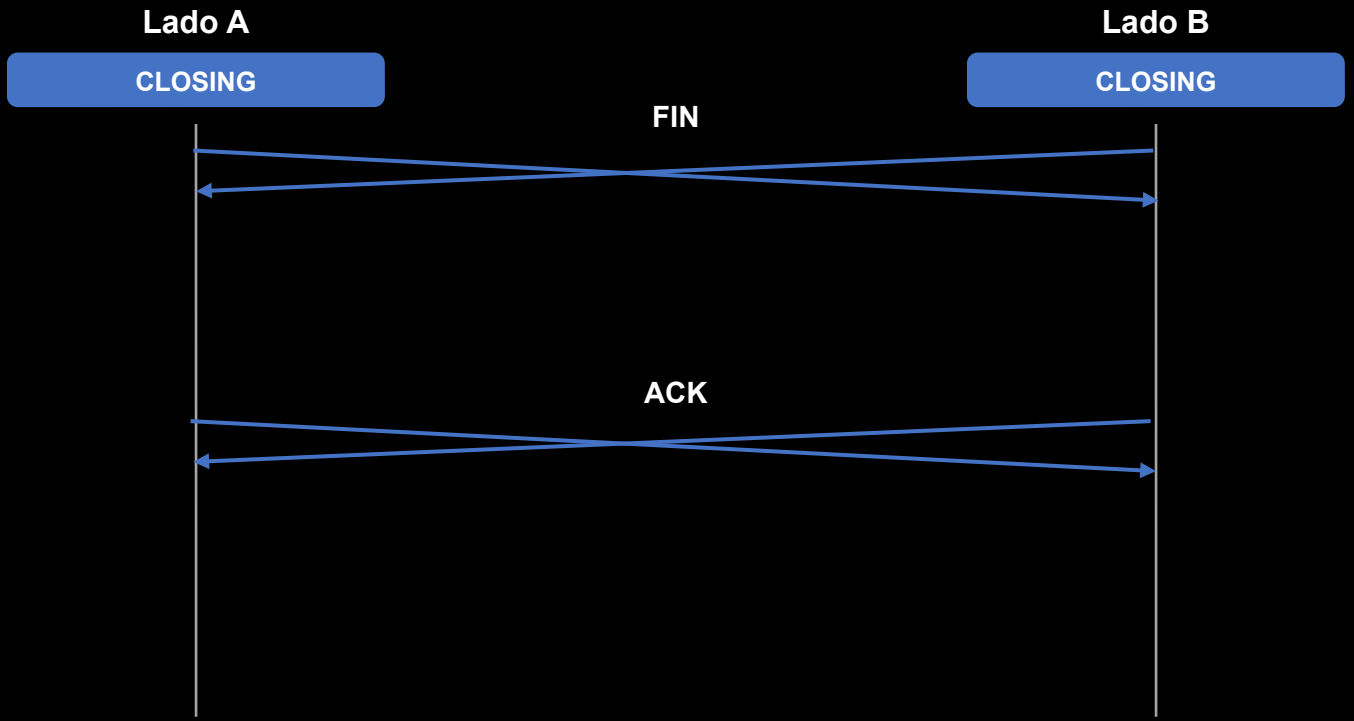
- Um fecho genuinamente simultâneo — raro, mas a máquina de estados do TCP tem de o consider na sua especificação: os dois FINs cruzam-se em trânsito antes de qualquer ACK chegar.

- ESTABLISHED → FIN_WAIT_1 → CLOSING → TIME_WAIT → CLOSED.

Ambos os Lados Fecham ao Mesmo Tempo



Ambos os Lados Fecham ao Mesmo Tempo



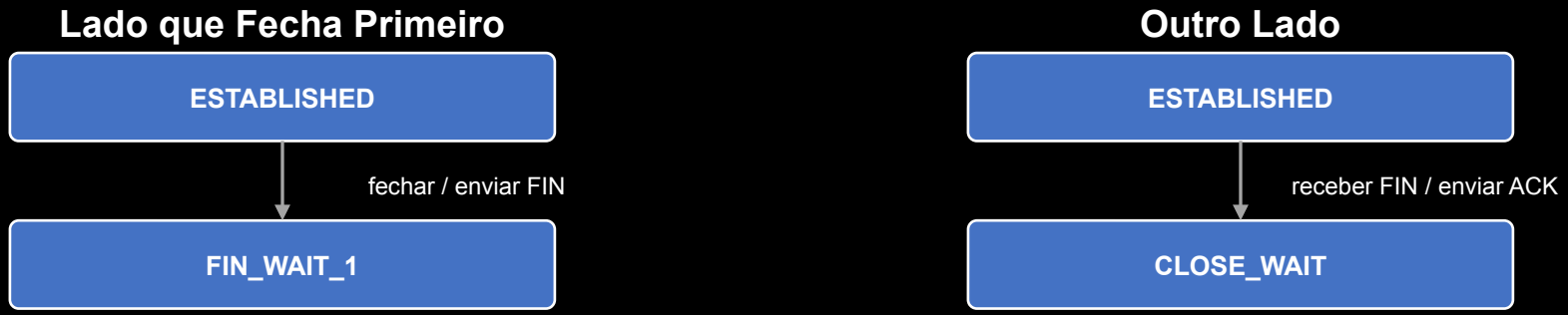
FIN_WAIT_1 vs. FIN_WAIT_2

- **FIN_WAIT_1:** este lado já enviou o seu próprio FIN, mas esse FIN ainda não foi confirmado — ainda há algo nosso em trânsito.
- **FIN_WAIT_2:** o nosso FIN já foi confirmado. Já não há nada nosso em trânsito — estamos agora apenas à espera do FIN do outro lado.
- **A diferença determina o que significa um FIN que chegue mais tarde:** um FIN que chegue enquanto ainda estamos em FIN_WAIT_1 significa que ambos os lados estão a fechar ao mesmo tempo (→ CLOSING); um FIN que chegue já estando em FIN_WAIT_2 significa apenas que o outro lado demorou mais a fechar (→ TIME_WAIT diretamente).
  - **Há ainda um raro quarto caminho:** um único segmento pode, por vezes, transportar ao mesmo tempo o ACK do nosso FIN e o próprio FIN do outro lado — saltando CLOSING por completo, diretamente de FIN_WAIT_1 para TIME_WAIT.

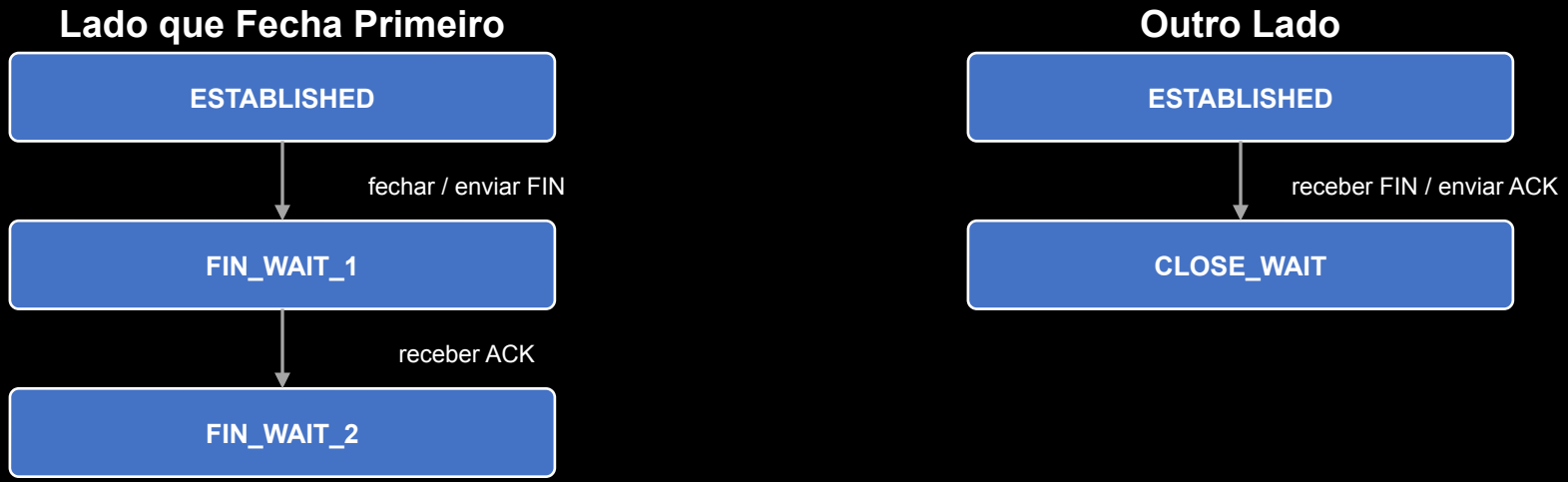
Os Três Caminhos, Lado a Lado

- **Eis o mesmo mecanismo, mas como uma máquina de estados mais clara —** dividida nas duas transições possíveis que de facto ocorrem (note-se, porém, que o TCP tem uma única máquina de estados que capta ambas as transições).
- **Um dos lados fecha primeiro:** cobre dois dos três caminhos ao mesmo tempo — seja qual for o lado que fecha primeiro, segue a forma da esquerda, o outro segue a da direita.
- **Ambos os lados fecham ao mesmo tempo:** um segundo diagrama, totalmente simétrico — o terceiro caminho.

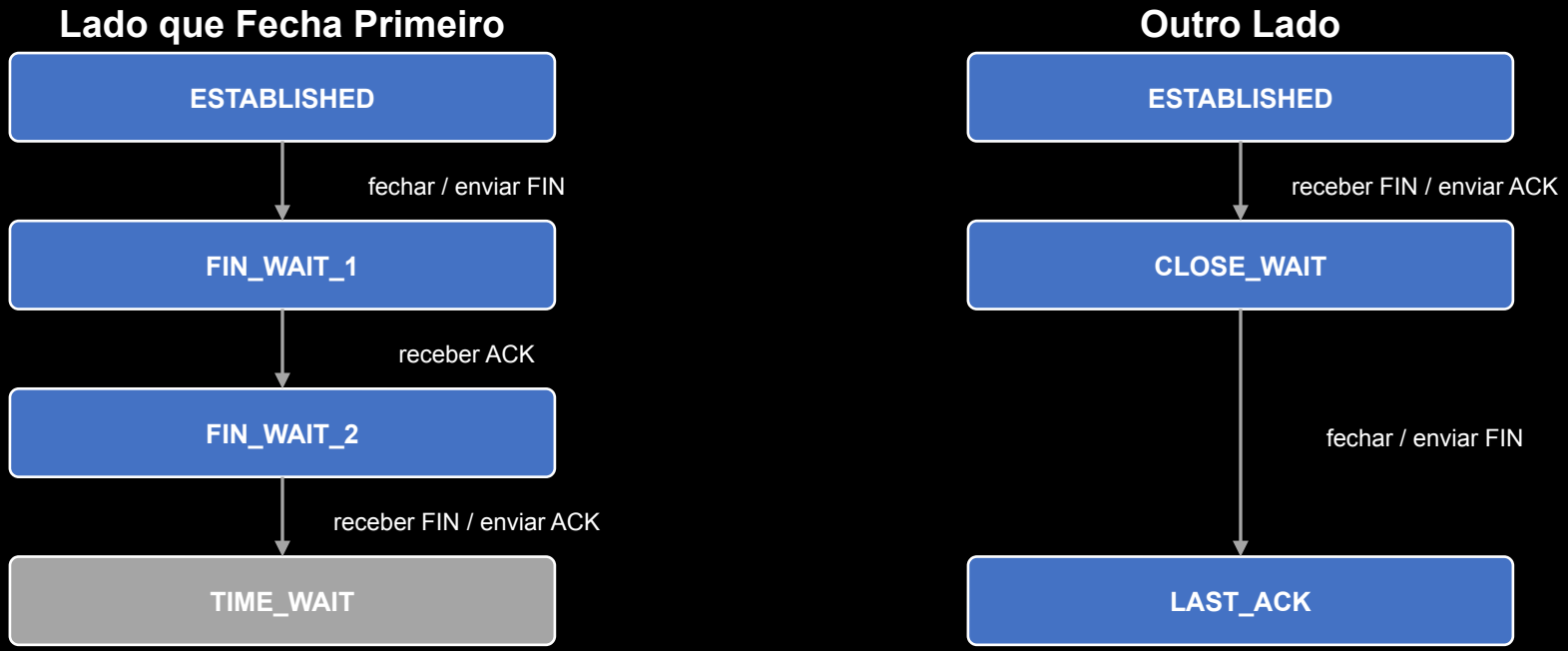
Um dos Lados Fecha Primeiro



Um dos Lados Fecha Primeiro



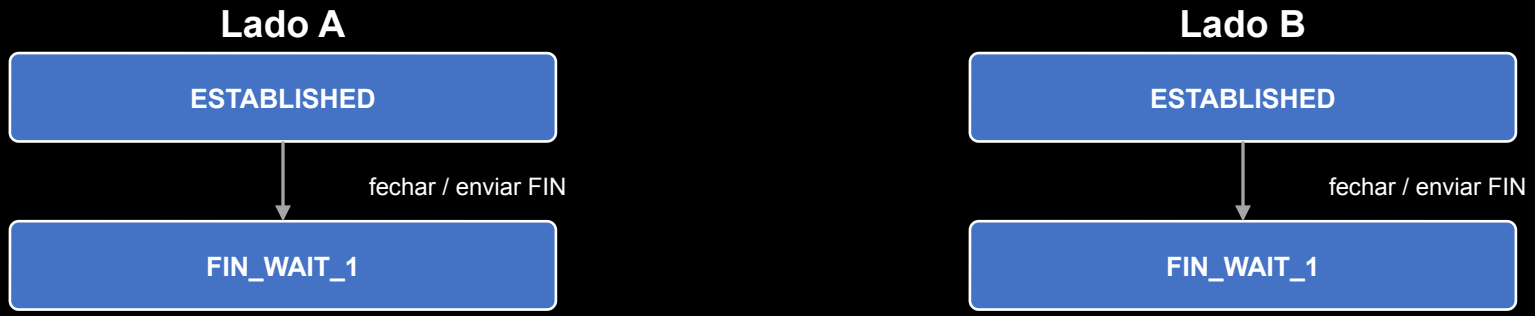
Um dos Lados Fecha Primeiro



Um dos Lados Fecha Primeiro



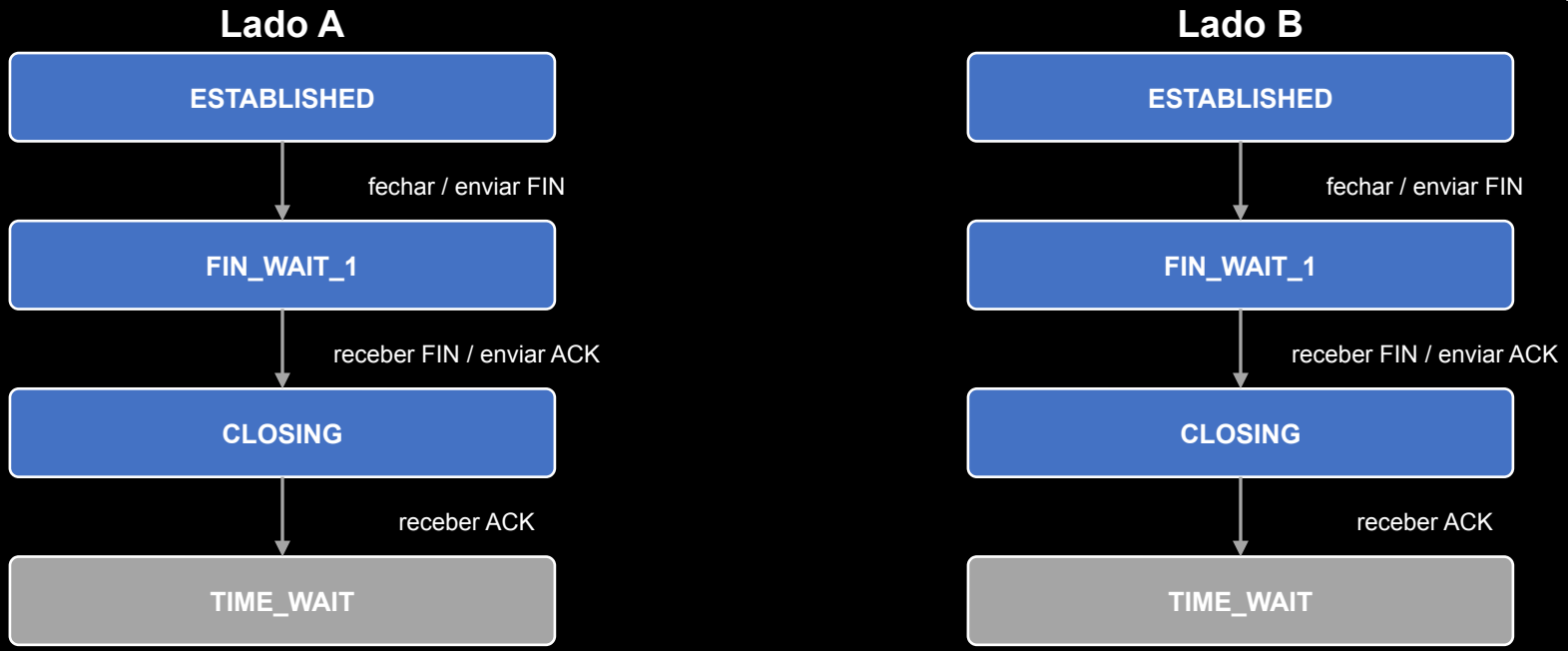
Ambos Fecham ao Mesmo Tempo



Ambos Fecham ao Mesmo Tempo



Ambos Fecham ao Mesmo Tempo



Ambos Fecham ao Mesmo Tempo



Porquê o TIME_WAIT?

- **Uma ligação não pode passar diretamente para o estado CLOSED** depois de enviar o ACK final — tem de esperar duas vezes o Maximum Segment Lifetime (2×MSL, cerca de 2×120s).
  - **A razão:** se esse ACK final se perder, o outro lado vai retransmitir o seu FIN — e se uma nova ligação já tivesse reaberto o mesmo par de portas, esse FIN perdido tentaria, imediata (e erradamente), derrubá-la.

- TIME_WAIT é o preço de garantir que o passado de uma ligação não se infiltra numa futura ligação com a mesma identidade.

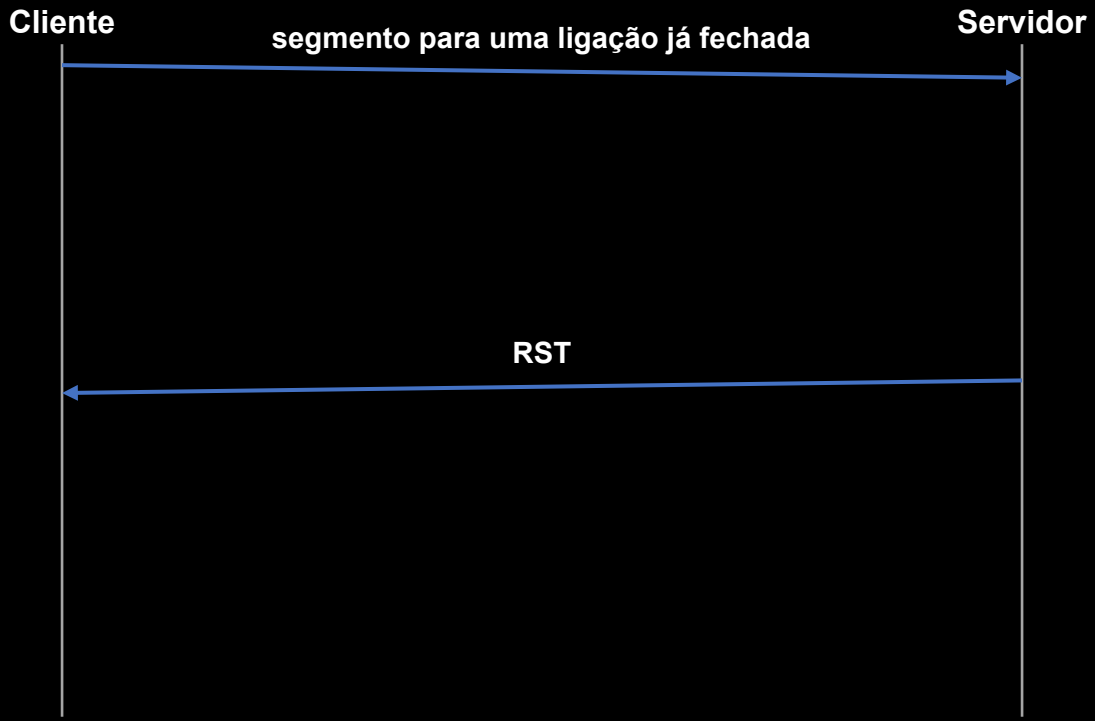
Ligar e Fechar

Conclusão:
o estabelecimento escolhe um único lado para fazer a abertura ativa; o encerramento não tem esse luxo — ambos os lados podem e fecham de forma independente, e é exatamente por isso que há três caminhos possíveis no diagrama de estados, e não apenas um.

Terminar Abruptamente: RST

- **Tudo até agora assumiu que ambos os lados cooperam para fechar de forma limpa —** quatro mensagens que permitem a ambos os lados concordar que a ligação pode ser fechada e não será usada novamente.
- **As redes reais nem sempre permitem isso.** RESET (RST) é a resposta do TCP quando não há forma possível de um fecho limpo.
- **O Reset (RST) dispara quando um lado recebe um segmento que não reconhece ou não quer —** por exemplo, um segmento para uma ligação já fechada, ou uma porta em que ninguém está à escuta — e quer abortar imediatamente.
- **Sem handshake, sem negociação, sem TIME_WAIT:** um único RST derruba a ligação no lado recetor de imediato — em contraste com o encerramento de quatro mensagens do FIN, com acordo de ambos os lados, que acabámos de ver.

RST: Uma Mensagem, Sem Negociação



Uma Coisa que Nenhum Handshake Resolve

Conclusão:
o RST aborta uma ligação de forma assimétrica e imediata — sem “despedida” envolvendo quatro mensagens, sem TIME_WAIT, apenas “parar agora”. Existe precisamente porque nem todas as ligações têm a oportunidade de fechar educadamente.

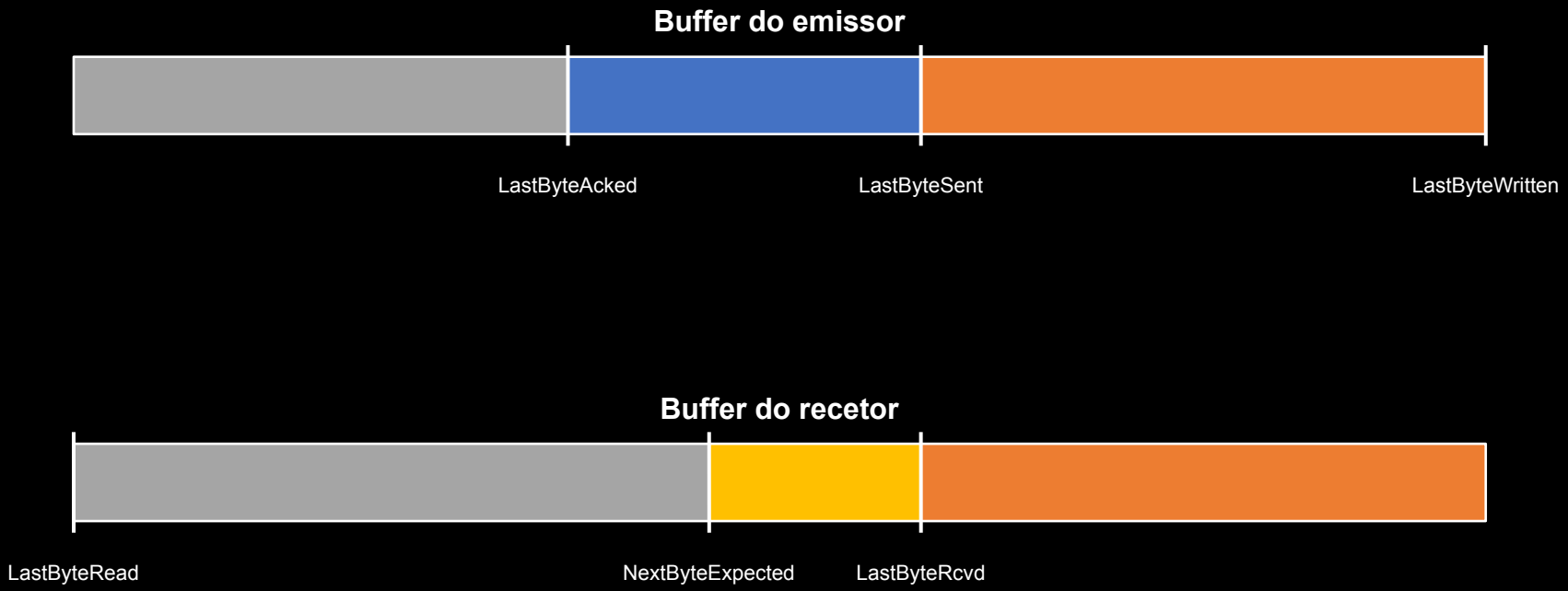
Mais Sobre a Janela Deslizante

- A janela deslizante já faz duas tarefas por nós: entrega fiável e entrega por ordem. Acontece que o mesmo mecanismo e a mesma informação de controlo conseguem fazer uma terceira tarefa também, quase sem esforço adicional.

Mais Sobre a Janela Deslizante

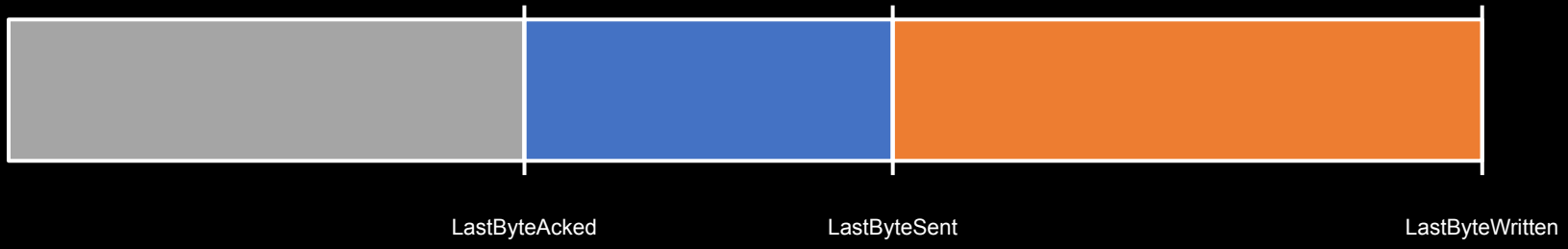
- A janela deslizante já faz duas tarefas por nós: entrega fiável e entrega por ordem. Acontece que o mesmo mecanismo e a mesma informação de controlo conseguem fazer uma terceira tarefa também, quase sem esforço adicional.
- **O TCP incorpora o controlo de fluxo:** impedindo que um emissor rápido sobrecarregue um recetor lento — usando exatamente o mesmo mecanismo de janela deslizante que já conhecemos.
- **A diferença em relação à versão ao nível da ligação física:** em vez de uma janela de tamanho fixo, o recetor anuncia ao emissor o seu próprio tamanho de janela, transportado no AdvertisedWindow campo que já vimos na semana passada.

Buffers de Envio e Receção



O Buffer do Emissor

Buffer do emissor

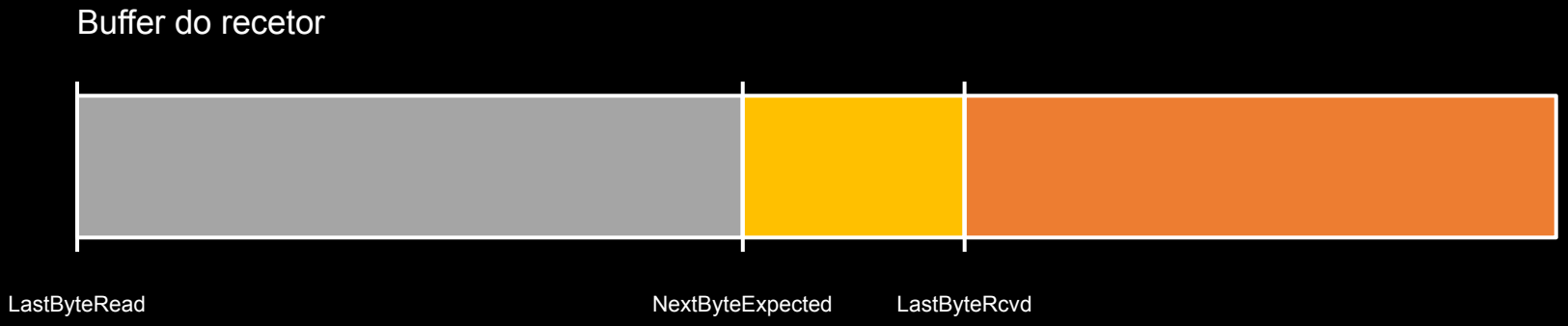


Contabilidade do Lado do Emissor

- LastByteAcked ≤ LastByteSent — o recetor não pode confirmar um byte que ainda não foi enviado.

- LastByteSent ≤ LastByteWritten — o TCP não pode enviar um byte que a aplicação ainda não escreveu.
  - Só os bytes entre LastByteAcked e LastByteWritten é que precisam de ficar em buffer.

O Buffer do Recetor

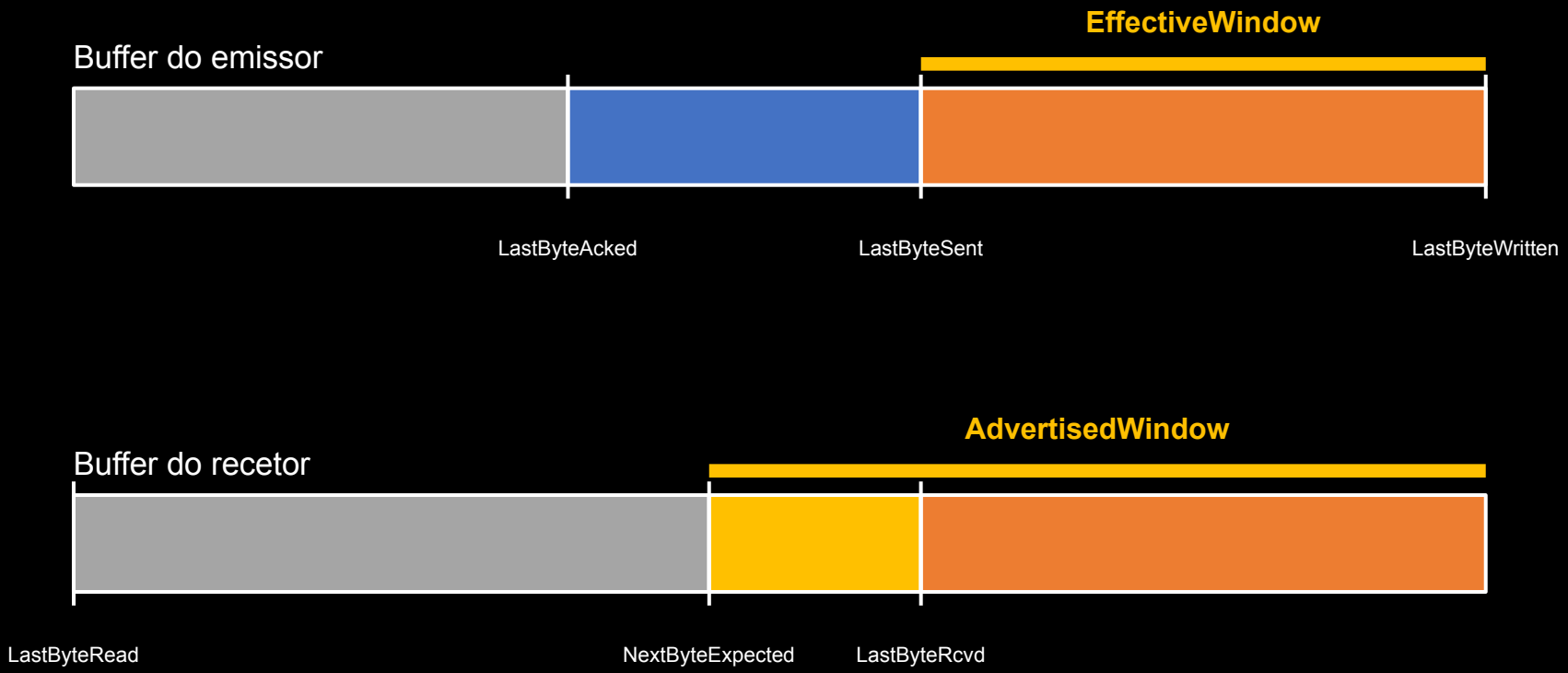


Contabilidade do Lado do Recetor

- LastByteRead < NextByteExpected — a aplicação não pode ler um byte até que ele (e tudo antes dele) tenha chegado.

- NextByteExpected ≤ LastByteRcvd + 1 — igual se os dados chegaram por ordem; apontando para a primeira lacuna, caso contrário.

Onde Vivem as Duas Janelas



AdvertisedWindow e EffectiveWindow

- **AdvertisedWindow = MaxRcvBuffer − ((NextByteExpected − 1) − LastByteRead):** o espaço livre no buffer do recetor, recalculado a cada segmento que este envia.

- **EffectiveWindow = AdvertisedWindow − (LastByteSent − LastByteAcked):** o que o emissor ainda está autorizado a transmitir, neste preciso momento.

- Se a aplicação recetora ficar atrasada, a janela anunciada encolhe — podendo chegar a 0 se o seu buffer ficar completamente cheio.

A Janela Encolhe

Buffer do recetor



Ainda há bastante espaço: AdvertisedWindow (dourado + laranja) ocupa a maior parte do buffer.

A Janela Encolhe

Buffer do recetor



A aplicação está a ficar atrasada — AdvertisedWindow está a encolher.

A Janela Encolhe

Buffer do recetor



Quase cheio. AdvertisedWindow está quase esgotada.

A Janela Encolhe

Buffer do recetor



AdvertisedWindow = 0. O emissor é instruído a parar por completo.

O Que Acontece em Zero?

- Uma janela de 0 significa que o emissor já não tem nada que esteja autorizado a enviar. Mas o TCP nunca envia dados sem ser solicitado — então, como é que o emissor chega a saber que a janela voltou a abrir (i.e., tem um valor maior que zero)?

- Vamos ver isso na próxima aula.

Resumo

- O modelo de ligação do TCP precisa de um three-way handshake explícito para estabelecer o estado partilhado — incluindo o número de sequência inicial de cada lado — antes de qualquer dado fluir.
- O encerramento é simétrico: cada lado fecha independentemente, dando origem a três caminhos possíveis no diagrama de estados, e a um TIME_WAIT obrigatório de 2×MSL para proteger a próxima encarnação da ligação — ou, abruptamente, um único RST, sem handshake.
- O controlo de fluxo reutiliza o mecanismo da janela deslizante: AdvertisedWindow permite ao recetor travar o emissor — podendo chegar a zero quando o buffer do recetor fica cheio.

Bibliografia

- Peterson, L. L., & Davie, B. S. (2019). Computer Networks: A Systems Approach (Release Version 6.1). Capítulo 2, Secção 2.5.2 — Sliding Window.
- Peterson, L. L., & Davie, B. S. (2019). Computer Networks: A Systems Approach (Release Version 6.1). Capítulo 5, Secção 5.2 — Reliable Byte Stream (TCP)

©2026 NOVA FCT